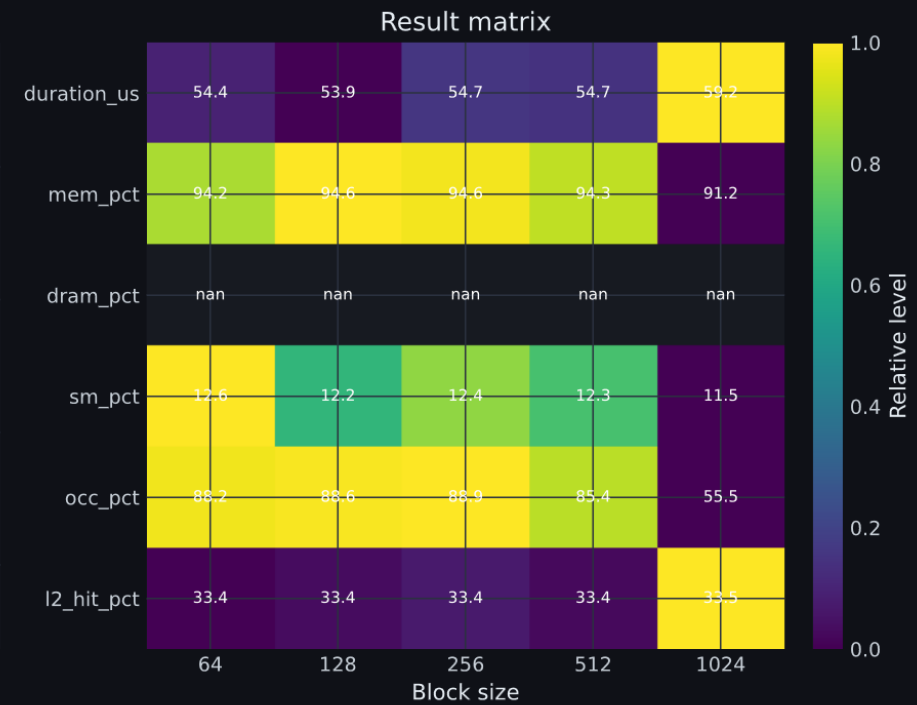
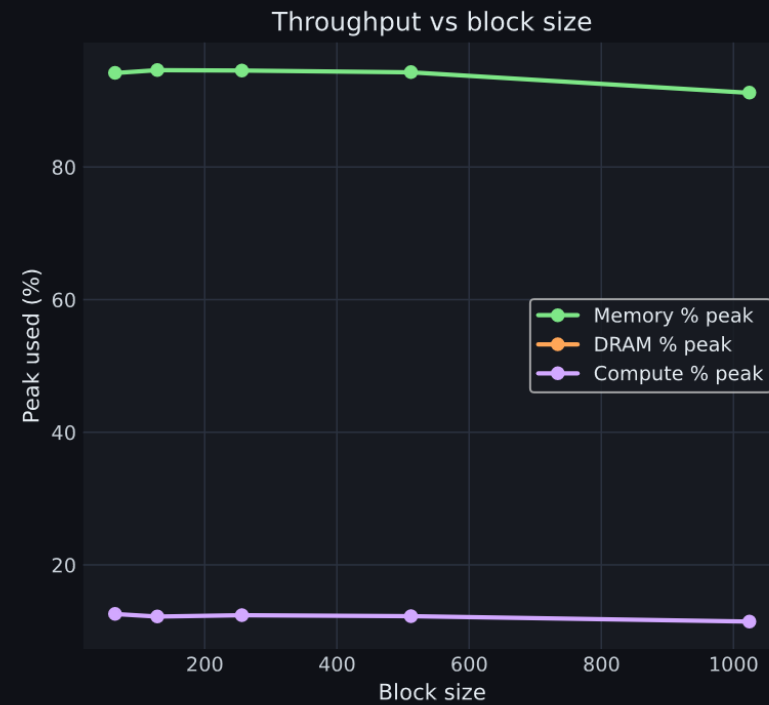
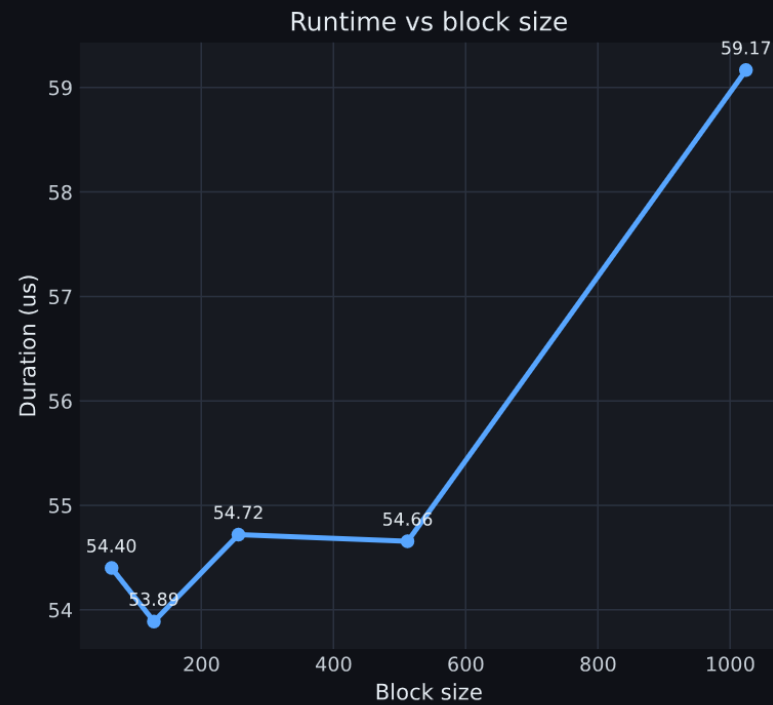


CUDA vecAdd block-size sweep



```
Collected metrics:
block_size grid_size duration_us mem_pct dram_pct sm_pct occ_pct l2_hit_pct regs_per_thread block_size_reported
64 16384.0 54.400 94.192459 NaN 12.599685 88.192341 33.431085 16.0 64.0
128 8192.0 53.888 94.628065 NaN 12.211601 88.583693 33.432184 16.0 128.0
256 4096.0 54.720 94.564943 NaN 12.410711 88.870512 33.433620 16.0 256.0
512 2048.0 54.656 94.303429 NaN 12.266916 85.401798 33.431932 16.0 512.0
1024 1024.0 59.168 91.208022 NaN 11.458184 55.464290 33.466198 16.0 1024.0
Fastest block size: 128 with 53.89 us, memory 94.63% peak, compute 12.21% peak.
```

👉 Block size: almost identical times for computation

👉 Memory percentage close 100%

Your GPU is doing the vector-add correctly, and the profiler says the GPU is spending most of its effort saturating memory bandwidth.

So the next optimization target would not be “more compute,” but either reducing memory traffic, improving reuse, or switching to a kernel with more arithmetic per byte moved.